

Data Structures and Algorithm

Semester III — Detailed Unit-wise Notes (Course Code: 2418301)

Unit 1.0 — Fundamentals of Algorithms and its Analysis

1.1 Data Types

A data type defines the kind of value a variable can hold and the operations that can be performed on it.

- System-defined (built-in) data types:** provided directly by the programming language, e.g., int, float, char, double, boolean. They have a fixed size and a predefined set of operations.
- User-defined data types:** created by the programmer to model real-world entities, e.g., structure, union, class, enumeration. They are built by combining one or more system-defined types.
- Data types are also classified as **primitive** (cannot be broken down further, e.g., int, char) and **non-primitive** (built from primitive types, e.g., arrays, structures, linked lists).

1.2 Basic Concept of Data Structure

A **data structure** is a systematic way of organizing, storing, and managing data so that it can be used efficiently.

- Linear data structure:** elements are arranged in a sequential order, and each element (except the first/last) has a unique predecessor and successor. Examples: array, linked list, stack, queue.
- Non-linear data structure:** elements are not arranged sequentially; each element may connect to multiple other elements. Examples: tree, graph.
- Abstract Data Type (ADT):** a logical/mathematical description of a data structure defined by the operations it supports, independent of how it is implemented. Example: a Stack ADT defines push/pop/peek without specifying whether it is implemented using an array or a linked list.

1.3 Algorithm and Its Analysis

An **algorithm** is a finite, well-defined, ordered sequence of steps that takes an input and produces a desired output to solve a problem.

Characteristics of a good algorithm: finiteness, definiteness, input/output well defined, effectiveness, and language independence.

Algorithm analysis estimates the resources (time and memory) an algorithm needs as a function of input size n .

- Time complexity:** the amount of computational time taken by an algorithm to run, expressed as a function of input size.
- Space complexity:** the amount of memory space required by an algorithm during execution (includes fixed part — instructions, constants — and variable part — variables, recursion stack).
- Worst-case analysis:** the maximum time/space taken for any input of size n ; gives an upper bound and is the most commonly used measure.
- Best-case analysis:** the minimum time/space taken for any input of size n ; gives a lower bound, usually less useful in practice.
- Average-case analysis:** the expected time/space averaged over all possible inputs of size n , weighted by their probability of occurrence.

1.4 Asymptotic Notation

Asymptotic notations describe the growth rate of an algorithm's running time as input size approaches infinity, ignoring machine-dependent constants.

- Big-O notation $O(g(n))$:** describes the **upper bound** (worst case) — $f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
- Omega notation $\Omega(g(n))$:** describes the **lower bound** (best case) — $f(n) = \Omega(g(n))$ if $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.
- Theta notation $\Theta(g(n))$:** describes a **tight bound** — used when the upper and lower bounds are of the same order, i.e., $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.
- Common growth-rate classes (best to worst): $O(1)$ constant, $O(\log n)$ logarithmic, $O(n)$ linear, $O(n \log n)$ linearithmic, $O(n^2)$ quadratic, $O(n^3)$ cubic, $O(2^n)$ exponential.

1.5 Time Complexity of Recursive Algorithm

Recursion occurs when a function calls itself, either directly or indirectly, to solve smaller instances of the same problem until a base case is reached.

- Every recursive algorithm has a **base case** (terminates recursion) and a **recursive case** (reduces the problem).
- The time complexity of a recursive algorithm is expressed using a **recurrence relation**, e.g., $T(n) = T(n-1) + O(1)$ for linear recursion.
- Master Theorem** is used to analyze recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$, $b > 1$:
- If $f(n) = O(n^{\log_b(a) - \epsilon})$, then $T(n) = \Theta(n^{\log_b(a)})$
- If $f(n) = \Theta(n^{\log_b(a)})$, then $T(n) = \Theta(n^{\log_b(a)} \cdot \log n)$
- If $f(n) = \Omega(n^{\log_b(a) + \epsilon})$ and the regularity condition holds, then $T(n) = \Theta(f(n))$
- Example: Binary search recurrence $T(n) = T(n/2) + O(1)$ gives $T(n) = O(\log n)$.

Unit 2.0 — Linear Data Structures

2.1 Array and String

An **array** is a collection of elements of the same data type stored in contiguous memory locations and accessed using an index.

- bullet **Single-dimensional array:** a simple linear list of elements, e.g., $A[0]$, $A[1]$, ..., $A[n-1]$.
- bullet **Multi-dimensional array:** an array of arrays, e.g., a 2-D array $A[i][j]$ used to represent matrices.
- bullet **Dynamic array:** an array that can resize itself automatically when it becomes full, typically by allocating a new larger array (often double the size) and copying old elements — amortized $O(1)$ insertion at the end.
- bullet **Array operations:** traversal $O(n)$, insertion $O(n)$ (shifting required), deletion $O(n)$ (shifting required), search — linear $O(n)$ or binary $O(\log n)$ if sorted, update/access $O(1)$.
- bullet **String:** a sequence of characters terminated by a null character (' $\backslash 0$ ') in C-like languages. Strings support operations such as concatenation, comparison, length calculation, substring extraction, and pattern searching. String manipulation includes reversing, case conversion, and tokenizing.

2.2 Linked List

A **linked list** is a linear data structure where each element (**node**) contains data and a pointer/reference to the next node, allowing dynamic memory allocation (unlike arrays, size is not fixed).

- bullet **Singly Linked List:** each node points only to the next node; the last node points to NULL. Traversal is possible only in the forward direction.
- bullet **Circular Linked List:** the last node points back to the first node, forming a circle. Useful for applications like round-robin scheduling, where continuous looping is required.
- bullet **Basic operations:**
 - bullet **Traversing:** visiting each node from head to the end, $O(n)$.
 - bullet **Insertion:** at the beginning $O(1)$, at the end $O(n)$ ($O(1)$ if tail pointer maintained), at a given position $O(n)$.
 - bullet **Deletion:** removing a node by adjusting the pointer of its predecessor to its successor, $O(n)$ in general, $O(1)$ if the node to delete is known and it's the head.
 - bullet **Modification:** changing the data value of a node after locating it.
- bullet Advantages over arrays: dynamic size, efficient insertion/deletion without shifting. Disadvantages: no random access, extra memory for pointers.

2.3 Stacks and Queue

Stack: a linear data structure following **LIFO (Last In First Out)** ordering — the most recently inserted element is removed first.

- bullet **Operations:** push (insert at top), pop (remove from top), peek/top (view top element), isEmpty, isFull — all $O(1)$.
- bullet **Implementation:** using a simple (fixed-size) array, a dynamic array, or a linked list (insertion/deletion at the head acts as push/pop).
- bullet **Applications:**
 - bullet Evaluating **postfix/infix expressions** — operators and operands are pushed/popped according to precedence rules.

bullet **Balancing symbols** (parentheses matching) — push opening brackets, pop and match on closing brackets.

bullet **Function calls** — the call stack stores return addresses and local variables for nested/recursive function calls.

Queue: a linear data structure following **FIFO (First In First Out)** ordering — the first element inserted is the first removed.

bullet **Operations:** enqueue (insert at rear), dequeue (remove from front), peek, isEmpty, isFull — all $O(1)$.

bullet **Implementation:** using a simple array (with front/rear pointers, may waste space), a circular array (reuses freed space), a dynamic array, or a linked list.

bullet **Applications:** CPU/disk scheduling, handling requests in order (printer queue), breadth-first search, buffering data streams.

Unit 3.0 — Non-Linear Data Structures

3.1 Tree

A **tree** is a hierarchical, non-linear data structure consisting of nodes connected by edges, with one node designated as the **root** and no cycles.

bullet **Terminology:**

bullet **Degree of a node** — number of children of that node.

bullet **Degree of tree** — maximum degree among all nodes.

bullet **Level of node** — distance (number of edges) from the root (root is at level 0).

bullet **Depth/Height of tree** — length of the longest path from root to a leaf.

bullet **In-degree/Out-degree** — number of incoming/outgoing edges of a node.

bullet **Path** — sequence of nodes/edges connecting two nodes.

bullet **Ancestor/Descendant** — any node on the path from root to a node is its ancestor; the node is its descendant.

bullet **Types of trees:**

bullet **Binary Tree:** each node has at most two children (left and right).

bullet **Binary Search Tree (BST):** a binary tree where, for every node, all values in the left subtree are smaller and all values in the right subtree are larger than the node's value — enables $O(\log n)$ average search/insert/delete.

bullet **Balanced tree:** a tree where the height difference between left and right subtrees of every node is bounded (e.g., AVL tree), keeping operations close to $O(\log n)$.

bullet **B-tree:** a self-balancing multi-way search tree commonly used in databases and file systems, where each node can have multiple keys/children, minimizing disk accesses.

bullet **Tree Traversal** (visiting all nodes):

bullet **In-order** (Left, Root, Right) — for a BST, gives nodes in sorted order.

bullet **Pre-order** (Root, Left, Right) — used to create a copy of the tree or prefix expressions.

bullet **Post-order** (Left, Right, Root) — used to delete a tree or evaluate postfix expressions.

3.2 Priority Queue and Heaps

A **priority queue** is an abstract data type in which each element has an associated priority; the element with the highest (or lowest) priority is served before others, regardless of insertion order.

bullet **Operations:** insert, delete-highest-priority (extract-max/min), peek, isEmpty, getSize.

bullet **Implementation using BST:** elements are stored maintaining BST order; the minimum/maximum priority element is the leftmost/rightmost node.

bullet **Heap:** a complete binary tree satisfying the **heap property**.

bullet **Min-Heap:** the value of each parent node is $\leq$ the values of its children; the minimum element is always at the root.

bullet **Max-Heap:** the value of each parent node is $\geq$ the values of its children; the maximum element is always at the root.

- bullet **Binary heap:** a heap implemented using an array, where for index i , children are at $2i+1$ and $2i+2$, and parent is at $(i-1)/2$.
- bullet **Operations:** insertion (add at end, then "bubble up" to restore heap property) $O(\log n)$; deletion of root (replace root with last element, then "heapify down") $O(\log n)$; peek $O(1)$.
- bullet **Heapify:** the process of converting an arbitrary array into a valid heap by repeatedly applying the sift-down/sift-up procedure, $O(n)$ overall for build-heap.

3.3 Graphs

A **graph** $G = (V, E)$ is a non-linear data structure consisting of a set of **vertices (V)** and a set of **edges (E)** connecting pairs of vertices.

- bullet **Basic terminology:**
- bullet **Weighted graph** — edges carry a numeric weight/cost; **unweighted graph** — edges have no associated cost.
- bullet **Directed graph (digraph)** — edges have direction; **undirected graph** — edges have no direction.
- bullet **Degree, in-degree, out-degree** — total/incoming/outgoing edges at a vertex.
- bullet **Articulation point** — a vertex whose removal increases the number of connected components (a "cut vertex"), important for network reliability analysis.
- bullet **Graph Representation:**
- bullet **Adjacency Matrix:** a $V \times V$ matrix where cell $(i,j) = 1$ (or weight) if an edge exists between vertex i and j , else $0/\infty$. Space $O(V^2)$; easy edge lookup $O(1)$.
- bullet **Adjacency List:** each vertex stores a list of its neighboring vertices. Space $O(V+E)$; efficient for sparse graphs.
- bullet **Adjacency Set:** similar to adjacency list but neighbors are stored in a set for $O(1)$ average edge-existence lookup and to avoid duplicates.
- bullet **Graph Traversal:**
- bullet **BFS (Breadth-First Search):** explores all neighbors at the current depth before moving deeper; uses a **queue**; useful for shortest path in unweighted graphs.
- bullet **DFS (Depth-First Search):** explores as far as possible along a branch before backtracking; uses a **stack** (or recursion); useful for cycle detection, topological sort.
- bullet **Shortest Path Algorithms:**
- bullet **Unweighted graph** — BFS gives the shortest path (fewest edges).
- bullet **Dijkstra's Algorithm** — finds the shortest path from a source to all vertices in a weighted graph with non-negative weights; uses a priority queue, complexity $O((V+E) \log V)$.
- bullet **Bellman-Ford Algorithm** — finds shortest paths even with negative edge weights and detects negative-weight cycles; complexity $O(V \cdot E)$.
- bullet **Floyd-Warshall Algorithm** — a dynamic-programming algorithm that finds shortest paths between all pairs of vertices in a weighted directed graph; complexity $O(V^3)$.
- bullet **Spanning Tree and Minimum Spanning Tree (MST):**
- bullet A **spanning tree** of a connected graph is a subgraph that includes all vertices with the minimum number of edges ($V-1$) and no cycles.
- bullet **Minimum Spanning Tree:** a spanning tree with the minimum possible total edge weight.
- bullet **Prim's Algorithm:** grows the MST one vertex at a time, always adding the cheapest edge connecting the tree to a new vertex; $O(E \log V)$ with a priority queue.

^{bullet} **Kruskal's Algorithm:** sorts all edges by weight and adds them one by one (using Union-Find to avoid cycles) until $V-1$ edges are included; $O(E \log E)$.

Unit 4.0 — Sorting and Searching Techniques

4.1 Sorting Techniques

Sorting arranges data in a particular order (ascending/descending).

- Bubble Sort:** repeatedly compares and swaps adjacent elements if they are in the wrong order, "bubbling" the largest element to the end each pass. Time complexity $O(n^2)$; simple but inefficient for large data.
- Selection Sort:** repeatedly selects the minimum (or maximum) element from the unsorted part and places it at the beginning. Time complexity $O(n^2)$; fewer swaps than bubble sort.
- Insertion Sort:** builds the sorted array one element at a time by inserting each new element into its correct position among the already-sorted elements. Time complexity $O(n^2)$ worst case, $O(n)$ best case (nearly sorted data); efficient for small/nearly sorted datasets.
- Quick Sort:** a divide-and-conquer algorithm that selects a **pivot**, partitions the array so elements smaller than the pivot go left and larger go right, then recursively sorts each part. Average time complexity $O(n \log n)$, worst case $O(n^2)$ (poor pivot choice); in-place and generally fast in practice.
- Merge Sort:** a divide-and-conquer algorithm that recursively divides the array into halves, sorts each half, and then **merges** the sorted halves. Time complexity $O(n \log n)$ in all cases; stable but requires $O(n)$ extra space.

4.2 Searching Techniques

- Linear Search:** examines each element of the list sequentially until the target is found or the list ends. Time complexity $O(n)$; works on unsorted data.
- Binary Search:** repeatedly divides a **sorted** array in half, comparing the target with the middle element and discarding the half where the target cannot lie. Time complexity $O(\log n)$; requires sorted data.

4.3 Hash Table

A **Hash Table** stores key-value pairs and provides near $O(1)$ average-time insertion, deletion, and lookup using a **hash function**.

- Hash Function:** maps a key to an index (or "bucket") in the underlying array; a good hash function distributes keys uniformly and minimizes collisions.
- Hash Collision:** occurs when two different keys map to the same index.
- Collision Resolution Techniques:**
 - Direct Chaining (Open Hashing):** each bucket holds a linked list (or other structure) of all elements hashing to that index; collisions simply append to the list.
 - Open Addressing:** on collision, the algorithm searches for the next available slot using a probing sequence — common methods are **linear probing** (check next slot sequentially), **quadratic probing** (check slots at quadratically increasing intervals), and **double hashing** (use a second hash function to determine the step size).

Unit 5.0 — Algorithm Design Techniques

5.1 Elements of Greedy Algorithm

A **greedy algorithm** builds a solution step by step, always choosing the option that looks best ("locally optimal") at the current moment, without reconsidering previous choices.

- Greedy choice property:** a globally optimal solution can be reached by making a sequence of locally optimal choices.
- Optimal substructure:** an optimal solution to the problem contains optimal solutions to its sub-problems.
- Huffman Coding Algorithm:** a greedy technique for **lossless data compression** — builds a binary tree where characters with lower frequency get longer codes and characters with higher frequency get shorter codes, by repeatedly merging the two least-frequent nodes. This minimizes the total encoded length.

5.2 Divide and Conquer Technique

The **Divide and Conquer** strategy solves a problem by:

1. **Divide** — breaking the problem into smaller sub-problems of the same type.
2. **Conquer** — solving the sub-problems recursively (directly if small enough).
3. **Combine** — merging the sub-problem solutions to form the solution to the original problem.

- Examples: Merge Sort, Quick Sort, Binary Search, finding maximum/minimum from n elements (compare pairs recursively, reducing comparisons compared to a linear scan).

5.3 Dynamic Programming Approaches

Dynamic Programming (DP) solves complex problems by breaking them into overlapping sub-problems, solving each sub-problem only once, and storing the results for reuse.

- Top-down approach (Memoization):** solve the problem recursively, but store ("memorize") the result of each sub-problem the first time it is computed, so repeated calls are answered in $O(1)$.
- Bottom-up approach (Tabulation):** solve the smallest sub-problems first and use their results to iteratively build up the solution to larger sub-problems, typically using a table/array.
- Overlapping sub-problems:** a property where the same sub-problem is solved multiple times in a naive recursive solution — DP exploits this by caching results.
- DP differs from divide-and-conquer in that DP sub-problems overlap (and are reused), while D&C sub-problems are independent.

5.4 Dynamic Programming Problems

- Longest Common Subsequence (LCS):** given two sequences, find the length of the longest subsequence present in both (not necessarily contiguous). Solved using a 2-D DP table where $dp[i][j]$ represents the LCS length of the first i and j characters of the two strings; time complexity $O(m \cdot n)$.
- Knapsack Problem (0/1 Knapsack):** given items with weights and values and a knapsack of fixed capacity, choose a subset of items maximizing total value without exceeding capacity. Solved using a DP table $dp[i][w]$ representing the maximum value achievable using the first i items within weight w ; time complexity $O(n \cdot W)$.

bullet **Matrix Chain Multiplication:** given a sequence of matrices, find the most efficient way (parenthesization) to multiply them together to minimize the total number of scalar multiplications. Solved using a DP table where $dp[i][j]$ stores the minimum cost of multiplying matrices i through j ; time complexity $O(n^3)$.